

Part 2: Notification Service — System Design

1. Overview

This document describes the architecture and design decisions for a scalable notification service capable of delivering notifications to millions of users across multiple channels, including **Email, SMS, and Push Notifications**.

The system is designed to support both:

- **Transactional notifications** — time-sensitive messages such as password resets, OTPs, payment confirmations, and security alerts.
- **Bulk notifications** — high-volume, lower-priority messages such as marketing campaigns, newsletters, and product announcements.

The architecture prioritises reliability, scalability, fault tolerance, message de-duplication, prioritisation, and maintainability.

2. High-Level Architecture

flowchart TB

A[Internal Services / Clients]

B[API Gateway / Load Balancer]

C[Notification API]

D[Authentication & Validation]

E[Priority Router]

F[Transactional Queue]

G[Bulk Queue]

H[Notification Workers]

I[Email Provider]

J[SMS Provider]

K[Push Provider]

L[(Notification Database)]

M[(Redis Cache / Idempotency Store)]

N[(Dead Letter Queue)]

O[Monitoring & Logging]

A --> B

B --> C

C --> D

D --> M

D --> L

C --> E

E -->|High Priority| F

E -->|Bulk / Low Priority| G

F --> H

G --> H

H --> I

H --> J

H --> K

H --> L

H --> M

F --> N

G --> N

C --> O

H --> O

N --> O

3. Architecture Components

3.1 API Gateway / Load Balancer

All internal services communicate with the notification service through an API Gateway or Load Balancer.

Responsibilities

- Distribute incoming requests across multiple API instances.
- Provide rate limiting.
- Enforce authentication and authorisation.
- Protect the service from traffic spikes.
- Improve availability by routing requests away from unhealthy instances.

Trade-off

A load balancer introduces additional infrastructure complexity, but it allows the API layer to scale horizontally and prevents a single server from becoming a bottleneck.

3.2 Notification API

The Notification API receives notification requests from other internal services.

Example request:

```
{  
  "notificationId": "abc-123",  
  "userId": "user-456",  
  "channel": "email",  
  "priority": "transactional",  
  "template": "password-reset",  
  "data": {  
    "name": "John",  
    "resetLink": "https://example.com/reset"  
  }  
}
```

The API should:

1. Authenticate the requesting service.
2. Validate the request.
3. Check for duplicate requests.
4. Store notification metadata.
5. Route the request to the appropriate queue.
6. Return quickly without waiting for the notification to be delivered.

Why asynchronous processing?

Sending an email or SMS can take seconds and depends on third-party providers. If the API waited synchronously, slow providers could reduce system performance.

By placing notifications into a queue, the API can acknowledge the request quickly while workers process delivery asynchronously.

4. Priority Routing

The service separates traffic into two categories.

Transactional Notifications

Examples include:

- Password resets
- OTP codes
- Payment confirmations
- Security alerts
- Account verification

These messages are time-sensitive and are placed in a **high-priority queue**.

Bulk Notifications

Examples include:

- Marketing campaigns
- Newsletters
- Promotional messages
- Product announcements

These messages are placed in a separate **bulk queue**.

Why separate queues?

If both types of traffic shared the same queue, a large marketing campaign could delay an important password reset email.

Separating the queues ensures that transactional traffic receives priority and maintains predictable latency.

Workers can also be independently scaled.

Transactional Queue → More workers during high-priority demand

Bulk Queue → Controlled worker capacity and lower priority

Trade-off

Multiple queues increase operational complexity, but they provide better isolation and prevent bulk workloads from affecting critical notifications.

5. Message Queue

A message broker is used between the Notification API and delivery workers.

Possible technologies include:

- RabbitMQ
- Apache Kafka
- AWS SQS

For this design, the service can use **RabbitMQ** or a managed queue such as **AWS SQS**, depending on the deployment environment.

Queue Responsibilities

The message queue:

- Decouples request processing from notification delivery.
- Absorbs traffic spikes.
- Enables asynchronous processing.
- Supports retries.
- Improves fault tolerance.

Trade-off

Queues provide resilience but introduce **eventual consistency**. A notification request being accepted does not necessarily mean it has already been delivered.

This trade-off is acceptable because the API can return a notification ID that clients can later use to check delivery status.

6. Notification Workers

Notification workers consume messages from the transactional and bulk queues.

Each worker:

1. Retrieves a notification message.
2. Checks the notification status.
3. Selects the appropriate delivery provider.
4. Sends the notification.
5. Updates the delivery status.
6. Records errors and retry attempts.

Workers can scale horizontally.

Queue

↓

Worker 1

Worker 2

Worker 3

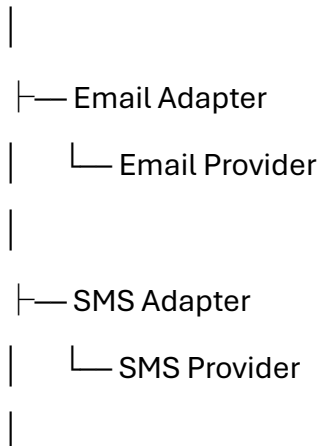
Worker N

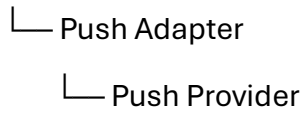
During periods of high traffic, additional worker instances can be started automatically.

7. Channel Providers

The system uses separate provider adapters for each communication channel.

Notification Worker





The adapter pattern prevents provider-specific logic from spreading throughout the application.

For example:

EmailProvider

SMSProvider

PushProvider

Each provider implements a common interface such as:

send(notification)

Benefits

- Providers can be replaced easily.
- New notification channels can be added.
- Failover providers can be introduced.
- Provider-specific code remains isolated.

8. Data Stores

8.1 Notification Database

A relational database stores persistent notification information.

Example fields:

Field	Description
notification_id	Unique identifier
user_id	Recipient
channel	Email, SMS, or Push
priority	Transactional or Bulk

Field	Description
status	Pending, Processing, Sent, Failed
retry_count	Number of delivery attempts
created_at	Creation timestamp
delivered_at	Successful delivery timestamp

A relational database is suitable because notification records have a clear structure and benefit from transactional updates.

Trade-off

Relational databases can become a bottleneck at extremely high write volumes. For larger-scale systems, notification logs may be partitioned by date or moved to specialised storage for analytics.

8.2 Redis Cache / Idempotency Store

Redis is used to prevent duplicate notification processing.

When a request arrives, the service checks whether the notificationId or idempotency key already exists.

Request

↓

Check Redis

↓

Already Exists?

└— Yes → Return existing result

└— No → Process and store key

The key is stored with a time-to-live.

Why Redis?

Redis provides extremely fast lookups and is well suited for short-lived idempotency records.

Trade-off

Redis is primarily an in-memory system, so persistent notification history should still be stored in the main database.

9. Retry Strategy

External providers can fail because of:

- Network problems
- Rate limits
- Temporary outages
- Provider timeouts

The service uses **exponential backoff** for retries.

Example:

Attempt 1 → Immediately

Attempt 2 → Wait 30 seconds

Attempt 3 → Wait 2 minutes

Attempt 4 → Wait 10 minutes

Attempt 5 → Wait 30 minutes

The exact retry policy can vary depending on the notification type.

Transactional notifications may have fewer but faster retries, while bulk notifications can tolerate longer delays.

Retry Flow

flowchart LR

A[Send Notification]

B{Successful?}

C[Mark as Sent]

D{Retry Limit Reached?}

E[Wait Using Exponential Backoff]

F[Dead Letter Queue]

A --> B

B -->|Yes| C

B -->|No| D

D -->|No| E

E --> A

D -->|Yes| F

Retries should only occur for **transient failures**.

Examples:

Retry:

- Timeout
- Temporary provider outage
- HTTP 429 rate limit
- HTTP 5xx server errors

Do not retry:

- Invalid phone number
- Invalid email address
- Unsupported channel

- Malformed request

This prevents the system from wasting resources retrying permanent failures.

10. Dead Letter Queue

Notifications that exceed the retry limit are moved to a **Dead Letter Queue (DLQ)**.

The DLQ stores messages that could not be delivered successfully.

Administrators can:

- Investigate the failure.
- Identify provider problems.
- Correct malformed data.
- Replay messages after the underlying issue is fixed.

The DLQ prevents permanently failing messages from blocking normal queue processing.

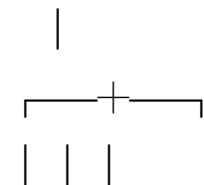
11. Failover Strategy

The system is designed to tolerate failures at multiple levels.

API Failover

Multiple Notification API instances run behind a load balancer.

Load Balancer



API API API

1 2 3

If one instance fails, traffic is routed to healthy instances.

Worker Failover

Multiple workers consume messages from the queues.

If a worker crashes before acknowledging a message, the message remains available for another worker to process.

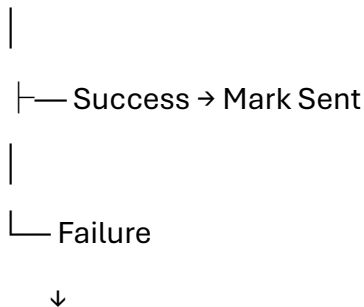
This provides fault tolerance without requiring a specific worker to remain operational.

Provider Failover

Where supported, the system can configure primary and secondary providers.

Example:

Primary Email Provider



Secondary Email Provider

Provider failover should only occur for failures indicating that the provider is unavailable, rather than permanent recipient errors.

Trade-off

Using multiple providers improves availability but increases:

- Cost
- Configuration complexity
- Operational monitoring requirements

For an initial implementation, provider failover may be limited to critical transactional traffic.

12. De-Duplication and Idempotency

Distributed systems can deliver the same message more than once.

For example:

1. A worker sends an email.
2. The provider successfully accepts it.
3. The worker crashes before recording success.
4. The queue redelivers the message.

Without protection, the user could receive duplicate notifications.

The system uses several layers of protection.

API-Level Idempotency

Each request includes a unique idempotency key.

```
{  
  "idempotencyKey": "payment-12345-receipt"  
}
```

If the same request is submitted multiple times, the service returns the original notification rather than creating another one.

Database Constraints

The notification ID is unique.

UNIQUE(notification_id)

This prevents duplicate notification records.

Worker-Level De-Duplication

Before sending a notification, the worker checks its current state.

Pending → Processing → Sent

If the notification is already marked as Sent, it should not be sent again.

Because complete exactly-once delivery cannot always be guaranteed when interacting with external providers, the system aims for **at-least-once processing with idempotent handling**.

This is more realistic and reliable than attempting to guarantee true exactly-once delivery across third-party systems.

13. Notification Lifecycle

stateDiagram-v2

[*] --> Pending

Pending --> Processing

Processing --> Sent

Processing --> Retry

Retry --> Processing

Retry --> Failed

Failed --> DeadLettered

Sent --> [*]

DeadLettered --> [*]

The lifecycle allows the system to track every notification and makes debugging easier.

14. Security Considerations

The notification service should implement the following security measures.

Authentication and Authorisation

Internal services authenticate using:

- API keys
- OAuth tokens
- Service-to-service authentication

Each service should only be allowed to perform authorised operations.

Data Protection

Sensitive information should be protected through:

- TLS encryption for data in transit.
- Encryption at rest for sensitive stored data.
- Secret management for API keys and provider credentials.

Secrets should never be stored directly in source code.

Input Validation

The API validates:

- Email addresses
- Phone numbers
- Supported channels
- Payload size
- Template parameters

This reduces malformed requests and protects the system from abuse.

Rate Limiting

Rate limits prevent:

- Accidental notification loops.
- Malicious abuse.
- A single internal service overwhelming the system.

Limits can be applied per service, user, or notification type.

15. Observability and Monitoring

The service should expose metrics including:

- Notifications received per second.
- Queue depth.
- Delivery success rate.
- Delivery latency.
- Retry count.
- Provider failure rate.
- Dead letter queue size.

Logs should include:

notification_id

user_id

channel

provider

status

retry_count

timestamp

Structured logging makes it easier to trace a notification throughout the system.

Alerts should be triggered when:

- Queue depth grows unexpectedly.
 - Provider failure rates increase.
 - DLQ messages exceed a threshold.
 - Transactional notification latency becomes too high.
-

16. Testing Strategy

The system should include multiple levels of testing.

Unit Tests

Test individual components such as:

- Request validation.
- Priority routing.
- Retry calculations.
- De-duplication logic.
- Provider adapters.

Integration Tests

Test interactions between:

- API and database.
- API and message queue.
- Workers and providers.
- Retry and DLQ behaviour.

External notification providers should be mocked during automated tests.

Load Tests

Load testing should simulate:

- Millions of notification requests.
- Large bulk campaigns.
- High transactional traffic.
- Provider failures.
- Worker failures.

The goal is to verify that transactional notifications continue receiving priority even when bulk queues experience heavy traffic.

17. Key Design Trade-offs

Decision	Benefit	Trade-off
Asynchronous queues	Improves scalability and resilience	Eventual consistency
Separate priority queues	Protects transactional traffic	More infrastructure complexity
Redis de-duplication	Fast idempotency checks	Additional infrastructure
Relational database	Strong consistency and structured data	May require partitioning at large scale
At-least-once processing	Reliable message processing	Requires de-duplication
Multiple providers	Better availability	Higher cost and complexity
Dead Letter Queue	Prevents poison messages blocking processing	Requires monitoring and operational procedures
Horizontal worker scaling	Handles traffic spikes	Increased infrastructure cost

18. Final Architecture Decision

The proposed architecture uses an **asynchronous, queue-based notification system** with independent API instances, priority queues, scalable workers, provider adapters, persistent notification tracking, and Redis-based idempotency.

The most important design decision is the separation of **transactional and bulk traffic**. This prevents large marketing campaigns from delaying critical notifications such as OTPs, password resets, and payment confirmations.

The system accepts that distributed delivery cannot always guarantee exactly-once delivery. Instead, it uses **at-least-once message processing combined with idempotency and de-duplication mechanisms** to minimise duplicate notifications while maintaining reliability.

Retries use exponential backoff, permanently failing messages are isolated in a Dead Letter Queue, and failures are handled through redundant API instances, multiple workers, and optional provider failover.

This architecture provides a balance between **reliability, scalability, operational complexity, and maintainability**, while allowing the notification service to grow from an initial implementation into a system capable of supporting millions of users.